

project3_598

Elizabeth Binkina

11/12/2025

```
library(dplyr)
```

```
##
```

```
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
```

```
##
```

```
##      filter, lag
```

```
## The following objects are masked from 'package:base':
```

```
##
```

```
##      intersect, setdiff, setequal, union
```

```
library(stringr)
```

```
movies_file <- "movies.dat"
```

```
ratings_file <- "ratings.dat"
```

```
users_file <- "users.dat"
```

```
read_movielens <- function(file, n_fields) {  
  lines <- readLines(file, warn = FALSE)  
  data <- str_split_fixed(lines, ":", n_fields)  
  as.data.frame(data, stringsAsFactors = FALSE)  
}
```

```
movies <- read_movielens(movies_file, 3)
```

```
ratings <- read_movielens(ratings_file, 4)
```

```
users <- read_movielens(users_file, 5)
```

```
colnames(movies) <- c("MovieID", "Title", "Genres")
```

```
colnames(ratings) <- c("UserID", "MovieID", "Rating", "Timestamp")
```

```
colnames(users) <- c("UserID", "Gender", "Age", "Occupation", "ZipCode")
```

```
movies$MovieID <- as.integer(movies$MovieID)
```

```
ratings$UserID <- as.integer(ratings$UserID)
```

```
ratings$MovieID <- as.integer(ratings$MovieID)
```

```
ratings$Rating <- as.numeric(ratings$Rating)
```

```
ratings$Timestamp <- as.numeric(ratings$Timestamp)
```

```
users$UserID <- as.integer(users$UserID)
```

```
# converting timestamp to readable date
ratings$Date <- as.POSIXct(ratings$Timestamp, origin = "1970-01-01")
```

```
# Merging datasets
movie_ratings <- merge(ratings, movies, by = "MovieID")

cat("Movies:", nrow(movies), "\n")
```

```
## Movies: 3883
```

```
cat("Ratings:", nrow(ratings), "\n")
```

```
## Ratings: 1000209
```

```
cat("Users:", nrow(users), "\n")
```

```
## Users: 6040
```

```
head(movie_ratings)
```

```
##   MovieID UserID Rating Timestamp          Date          Title
## 1      1    3900      5 965875114 2000-08-09 21:38:34 Toy Story (1995)
## 2      1    4561      5 964497430 2000-07-24 22:57:10 Toy Story (1995)
## 3      1    3113      4 969466395 2000-09-20 11:13:15 Toy Story (1995)
## 4      1    3403      3 967427541 2000-08-27 20:52:21 Toy Story (1995)
## 5      1    3841      2 965996269 2000-08-11 07:17:49 Toy Story (1995)
## 6      1     739      5 975475262 2000-11-28 23:21:02 Toy Story (1995)
##                                     Genres
## 1 Animation|Children's|Comedy
## 2 Animation|Children's|Comedy
## 3 Animation|Children's|Comedy
## 4 Animation|Children's|Comedy
## 5 Animation|Children's|Comedy
## 6 Animation|Children's|Comedy
```

Part A

“Popularity” definition: based on average rating among movies with at least 1250 user ratings

```
movie_summary <- movie_ratings %>%
  group_by(MovieID, Title) %>%
  summarise(
    avg_rating = mean(Rating),
    num_ratings = n()
  ) %>%
  ungroup()
```

```
## ‘summarise()’ has grouped output by ‘MovieID’. You can override using the
## ‘.groups’ argument.
```

```
# Top 10 Highest Rated (with >= 1250 ratings)
```

```
top_highest_rated <- movie_summary %>%
  filter(num_ratings >= 1250) %>%
  arrange(desc(avg_rating)) %>%
  head(10)
```

```
cat("\nTop 10 Highest Rated Movies :\n")
```

```
##
```

```
## Top 10 Highest Rated Movies :
```

```
print(top_highest_rated)
```

```
## # A tibble: 10 x 4
```

```
##   MovieID Title                                avg_rating num_ratings
##   <int> <chr>                                <dbl>         <int>
## 1    318 Shawshank Redemption, The (1994)      4.55           2227
## 2    858 Godfather, The (1972)                 4.52           2223
## 3     50 Usual Suspects, The (1995)            4.52           1783
## 4    527 Schindler's List (1993)               4.51           2304
## 5   1198 Raiders of the Lost Ark (1981)         4.48           2514
## 6    260 Star Wars: Episode IV - A New Hope (1977) 4.45           2991
## 7    750 Dr. Strangelove or: How I Learned to Stop Wor~ 4.45           1367
## 8    912 Casablanca (1942)                     4.41           1669
## 9   2762 Sixth Sense, The (1999)               4.41           2459
## 10   1193 One Flew Over the Cuckoo's Nest (1975) 4.39           1725
```

B

```
library(tidyr)
library(tibble)
library(stringr)
```

```
R_dense <- ratings %>%
  select(UserID, MovieID, Rating) %>%
  mutate(UserID = as.integer(UserID),
         MovieID = as.integer(MovieID)) %>%
  arrange(UserID, MovieID) %>%
  pivot_wider( names_from = MovieID, values_from = Rating, values_fill = NA_real_
  ) %>%
  arrange(UserID)
```

```
user_ids <- R_dense$UserID
```

```
R_dense <- R_dense %>% remove_rownames() %>% column_to_rownames("UserID")
```

```
R_dense <- as.matrix(R_dense)
```

```
row_means <- rowMeans(R_dense, na.rm = TRUE)
```

```
R_centered_dense <- sweep(R_dense, 1, row_means,
  FUN = function(x, m) ifelse(is.na(x), NA_real_, x - m)
)
```

```
movie_ids <- as.integer(colnames(R_dense))
cat("\nCentered rating matrix dimensions:\n")
```

```
##
## Centered rating matrix dimensions:
```

```
dim(R_centered_dense)
```

```
## [1] 6040 3706
```

2.

```
library(Matrix)
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
R_filled <- R_centered_dense
R_filled[is.na(R_filled)] <- 0
R_sparse <- Matrix(R_filled, sparse = TRUE)

co_counts <- as.matrix((t(!is.na(R_centered_dense)) %*% (!is.na(R_centered_dense))))
num <- as.matrix(t(R_sparse) %*% R_sparse)
norms <- sqrt(colSums(R_sparse^2))
denom <- outer(norms, norms)
cosine_raw <- num / denom

cosine_raw[co_counts < 3] <- NA
cosine_sim <- (1 + cosine_raw) / 2
diag(cosine_sim) <- 1

cat("\nSimilarity matrix dimensions:", dim(cosine_sim), "\n")
```

```
##
## Similarity matrix dimensions: 3706 3706
```

```
cat("\nSimilarity Matrix ")
```

```
##
## Similarity Matrix
```

```
print(cosine_sim[1:5, 1:5])
```

```
##           1           48           150           260           527
## 1  1.0000000 0.4458159 0.6081202 0.6416585 0.6240017
## 48 0.4458159 1.0000000 0.4884758 0.4343243 0.4437413
## 150 0.6081202 0.4884758 1.0000000 0.6030513 0.6294177
## 260 0.6416585 0.4343243 0.6030513 1.0000000 0.6622754
## 527 0.6240017 0.4437413 0.6294177 0.6622754 1.0000000
```

3.

```
library(dplyr)
library(stringdist)
```

```
## Warning: package 'stringdist' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'stringdist'
```

```
## The following object is masked from 'package:tidyr':
```

```
##
```

```
##      extract
```

```
keep_top_k <- function(sim_matrix, k = 30) {
  n <- nrow(sim_matrix)
  sim_filtered <- matrix(NA_real_, n, n, dimnames = dimnames(sim_matrix))
  for (i in seq_len(n)) {
    sims <- sim_matrix[i, ]
    sims[i] <- NA
    valid_idx <- which(!is.na(sims))
    if (length(valid_idx) > k) {
      top_k <- order(sims[valid_idx], decreasing = TRUE)[1:k]
      keep_idx <- valid_idx[top_k]
    } else {
      keep_idx <- valid_idx
    }
    sim_filtered[i, keep_idx] <- sims[keep_idx]
  }

  sim_filtered <- pmax(sim_filtered, t(sim_filtered), na.rm = TRUE)
  diag(sim_filtered) <- 1
  return(sim_filtered)
}

cosine_top30 <- keep_top_k(cosine_sim, k = 30)
movie_lookup <- movies %>%
  mutate(MovieID = as.character(MovieID),
         Title = iconv(Title, from = "", to = "UTF-8"))
}
```

```

target_titles <- c("Toy Story (1995)", "GoldenEye (1995)", "Liar Liar (1997)", "Lost World: Jurassic Park")
target_titles <- iconv(target_titles, from = "", to = "UTF-8")
matched_movie_ids <- sapply(target_titles, function(t) {
  movie_lookup$MovieID[
    which.min(stringdist(t, movie_lookup$Title, method = "jw"))
  ]
})

names(matched_movie_ids) <- target_titles

cat("\n TOP-30 filtered pairwise similarity values \n\n")

```

```

##
## TOP-30 filtered pairwise similarity values

```

```

for (i in seq_along(matched_movie_ids)) {
  movie_id <- matched_movie_ids[i]
  movie_title <- names(matched_movie_ids)[i]
  sims <- cosine_top30[movie_id, ]
  sims_non_na <- sims[!is.na(sims) & names(sims) != movie_id]
  cat(movie_title, ":\n")
  print(sims_non_na)
  cat("\n")
}

```

```

## Toy Story (1995) :
##      150      260      527      588      595      720      745      919
## 0.6081202 0.6416585 0.6240017 0.6363491 0.6228407 0.5857721 0.6174270 0.6159848
##      1028      1029      1035      1097      1197      1270      1907      2028
## 0.5903668 0.5588177 0.5603516 0.6091406 0.6289569 0.6396530 0.5931428 0.6245649
##      2321      2355      2687      2762      2797      2804      2918      3114
## 0.5606816 0.6508083 0.5530621 0.6257803 0.6061375 0.6211405 0.6070074 0.7514795
##      318      356      457      480      589      593      1196      1198
## 0.6437938 0.6181323 0.6161459 0.5761817 0.6044159 0.6139623 0.6428333 0.6553951
##      1210      1259      1265      1784      1968      2396      2501      2571
## 0.6035714 0.5986379 0.5977776 0.5804041 0.5820232 0.5963533 0.5747279 0.6257652
##      3147      3255      1079      1291      1580      1641      2081      1036
## 0.5821785 0.5624883 0.5832397 0.6209660 0.5728714 0.5640300 0.5890190 0.6096787
##      34      39      497      1500      1653      2580      2716      2987
## 0.6043058 0.5422171 0.5778940 0.5605439 0.5416248 0.5493375 0.5933273 0.5656565
##      3793      17      364      474      1411      1682      1148      1223
## 0.5562029 0.5642067 0.6279620 0.5695214 0.5508125 0.5881835 0.6283995 0.5978763
##      1307      1356      2302      3178      3751      3916      1023      1073
## 0.5995447 0.5453179 0.5752825 0.5430552 0.5732661 0.5485042 0.5640920 0.5675991
##      1374      1376      2078      2080      2137      3175      3189      440
## 0.5733915 0.5457088 0.5847561 0.5708651 0.5697335 0.5706724 0.5307039 0.5642122
##      3396      1396      2761      3397      3526      1407      3000      3044
## 0.6061992 0.5516116 0.5954408 0.5543440 0.5524104 0.5460288 0.5587414 0.5525205
##      2144      3361      3159      3429      634      1871      776      823
## 0.5434415 0.5588935 0.5633247 0.5759063 0.4889731 0.5113307 0.4980693 0.5004494
##      1532
## 0.5005925
##

```

```

## GoldenEye (1995) :
##      163      165      349      368      380      457      648      780
## 0.5595606 0.5732000 0.5527713 0.5459082 0.5756609 0.5518999 0.5507003 0.5641898
##      1210     1370     1527     2006     2353     2490     2628     3107
## 0.5514261 0.5563893 0.5434722 0.5686859 0.5518383 0.5425422 0.5481280 0.5606704
##      3256      733      1291     1580     2115     2617     2947     353
## 0.5590288 0.5636427 0.5622491 0.5612744 0.5450701 0.5414998 0.5643171 0.5473331
##      1485     1722      861     1573      367     2001     2948     2414
## 0.5446944 0.6811348 0.5502504 0.5505785 0.5460743 0.5714669 0.5804118 0.5419553
##      832      1805     2273     2763     3638     2989     2993     3633
## 0.5501348 0.5391586 0.5447734 0.5374702 0.5570615 0.5932092 0.6000820 0.5676936
##      3635     3639     2376     2949     2990     2991     3082      145
## 0.5889726 0.6102668 0.5645636 0.5902305 0.5787683 0.5898952 0.6342587 0.5438540
##      799      3763     3555     2476      251     2984
## 0.5459792 0.5335576 0.5326196 0.5468349 0.5429733 0.5238562
##
## Liar Liar (1997) :
##      380     1917     3809      104     2115     1923     474      367
## 0.5505851 0.5461005 0.5565123 0.5725462 0.5440695 0.5499154 0.5422749 0.5685531
##      1777       62      333      344      586     2001     2011     2423
## 0.5685403 0.5444561 0.5851386 0.5955287 0.5519291 0.5582037 0.5518728 0.5687311
##      2496     3087      216      231      440     2335     2706      10
## 0.5461372 0.5570468 0.5623236 0.5912342 0.5448908 0.5557770 0.5421977 0.5446944
##      2414      500      832     1689     3450     450      472     3640
## 0.5359123 0.5676886 0.5437210 0.5512704 0.5314250 0.5505013 0.5493251 0.5236784
##      2474      634     1567
## 0.5364930 0.4936222 0.5453606
##
## Lost World: Jurassic Park (1997) :
##      48      95      292     434     648     736     780     1552
## 0.5557256 0.5825291 0.5730773 0.5779968 0.5531375 0.6017312 0.5809405 0.5857165
##      1690     1917     2628     2617     2688     3173     367     3623
## 0.5714233 0.6127306 0.5607157 0.5392223 0.5496186 0.5415758 0.5485967 0.5503164
##      3755     3826      153      208      316      329     586     1320
## 0.5359067 0.5786915 0.5964871 0.5990836 0.5657593 0.5774918 0.5607038 0.5796582
##      1371     1377     1676     1831     2393     2399     788      70
## 0.5616441 0.5621398 0.5552876 0.6162065 0.5627255 0.5581628 0.5518310 0.5563827
##      160      748     1608     2433     2701     1779     3638     1562
## 0.5966922 0.5491473 0.5610087 0.5328902 0.5768922 0.5903933 0.5631441 0.6145603
##      1876     1882     2402     1373      173      185     786     849
## 0.5755064 0.6114219 0.5708603 0.5976255 0.5829555 0.5809324 0.5706421 0.5727345
##      1037     1438      196     2642       3     1556     494     1894
## 0.5772170 0.5762721 0.5820406 0.5756457 0.5535593 0.5724340 0.5471345 0.5691633
##      1102       22      634     3825     3754     3817     2821     2887
## 0.5050519 0.5497427 0.5007714 0.5538385 0.5419945 0.5187225 0.5109865 0.5135413
##      3136     3876
## 0.4950586 0.5226489
##
## Sixth Sense, The (1999) :
##      1      150      260      527     1197     1246     1270     1961
## 0.6257803 0.6027735 0.6654567 0.6538433 0.6207843 0.5760715 0.6165879 0.5933417
##      2028     2321     2355     2692     2804     2918     3114     3408
## 0.6616607 0.5494938 0.5719495 0.5927457 0.5969360 0.6030039 0.6204764 0.5678458
##      110      318      356      457      480      589      590      593

```

```
## 0.6384163 0.6919740 0.6290014 0.6457116 0.5825435 0.6151535 0.5766294 0.6836017
##      1196      1198      1210      1259      1265      1408      1610      1784
## 0.6525759 0.6717019 0.6131732 0.5995186 0.5906618 0.5523945 0.6019771 0.5787811
##      1968      2268      2396      2501      2571      2858      3147      3578
## 0.5732268 0.5880821 0.6100318 0.5648439 0.6714809 0.6462496 0.6404246 0.6360559
##      1291      1580      1641      1036      1214      1240      47      50
## 0.6251416 0.5624883 0.5595239 0.6237424 0.6076082 0.6292192 0.6223008 0.6827600
##      296      1500      1617      1704      1923      2058      2580      2716
## 0.6318183 0.5616942 0.6296727 0.6348453 0.5933200 0.5434824 0.5741144 0.6012096
##      2959      3006      3793      364      1674      3536      474      1393
## 0.5910730 0.5720143 0.5847715 0.5723020 0.5749085 0.5388174 0.5698214 0.5653394
##      1682      2324      2329      2336      2686      3148      3252      223
## 0.5790130 0.6016592 0.6029757 0.5744621 0.5525873 0.5605834 0.5426345 0.5821073
##      529      1358      1584      3178      3510      3916      3948      539
## 0.5698032 0.6092451 0.5493763 0.5599184 0.5650807 0.5562200 0.5328565 0.5497291
##      858      1200      2000      3175      2706      3897      553      628
## 0.6287637 0.6064974 0.5838336 0.5494684 0.5490948 0.5896733 0.5480343 0.5794939
##      1396      1625      2841      293      1242      2502      3052      3044
## 0.5575558 0.5857959 0.5478519 0.5875692 0.6153971 0.5553577 0.5529690 0.5547629
##      1680      2679      1877      853      696      1886      3242      3609
## 0.5464387 0.5023186 0.5003761 0.5027409 0.5076507 0.4992191 0.5046407 0.5024099
##      404
## 0.5079583
```

4.

```
IBCF_recommend <- function(newuser,sim_matrix,movie_summary,top_n = 10,
                             popularity_ranking_file = "movie_popularity.csv") {

  if (length(newuser) != ncol(sim_matrix))
    stop("")

  user_mean <- mean(newuser, na.rm = TRUE)
  centered_user <- newuser - user_mean
  rated_idx <- which(!is.na(centered_user))
  unrated_idx <- which(is.na(centered_user))
  preds <- rep(NA_real_, length(newuser))

  for (j in unrated_idx) {
    sims <- sim_matrix[j, rated_idx]
    ratings <- centered_user[rated_idx]
    valid <- !is.na(sims)
    if (sum(valid) >= 1) {
      num <- sum(sims[valid] * ratings[valid], na.rm = TRUE)
      denom <- sum(abs(sims[valid]), na.rm = TRUE)
      if (denom > 0) {
        preds[j] <- user_mean + (num / denom)
      }
    }
  }

  names(preds) <- colnames(sim_matrix)
```



```

pred_df <- data.frame(Title = names(preds), PredictedRating = preds
) %>%
  filter(!is.na(PredictedRating)) %>% arrange(desc(PredictedRating))

if (file.exists(popularity_ranking_file)) {
  pop_rank <- read.csv(popularity_ranking_file, stringsAsFactors = FALSE)
} else {
  pop_rank <- movie_summary %>%
    arrange(desc(num_ratings), desc(avg_rating)) %>%
    select(Title)
  write.csv(pop_rank, popularity_ranking_file, row.names = FALSE)
}

already_rated_titles <- names(newuser)[which(!is.na(newuser))]

if (nrow(pred_df) < top_n) {
  needed <- top_n - nrow(pred_df)
  fillers <- pop_rank %>%
    filter(!(Title %in% pred_df$Title),
            !(Title %in% already_rated_titles)) %>%
    slice_head(n = needed) %>%
    mutate(PredictedRating = NA)
  pred_df <- bind_rows(pred_df, fillers)
}

top_recs <- pred_df %>%
  slice_head(n = top_n)

return(top_recs)
}

```

5.

```

library(dplyr)
library(stringdist)

user1500 <- R_dense[1500, ]
names(user1500) <- colnames(R_dense)

cat("\nTop 10 Recommendations for Existing User (row 1500)\n")

##
## Top 10 Recommendations for Existing User (row 1500)

rec_user1500 <- IBCF_recommend(
  newuser      = user1500,
  sim_matrix    = cosine_top30,
  movie_summary = movie_summary,
  top_n         = 10
)

```

```
print(rec_user1500)
```

```
##      Title PredictedRating
## 594    594                5
## 1029   1029               5
## 1907   1907               5
## 2018   2018               5
## 2687   2687               5
## 902    902                5
## 1293   1293               5
## 1953   1953               5
## 2278   2278               5
## 2427   2427               5
```

```
newuser <- rep(NA, ncol(cosine_top30))
names(newuser) <- colnames(cosine_top30)

starwars_id <- "260"
id4_id <- "780"

newuser[starwars_id] <- 5
newuser[id4_id] <- 4

cat("\nTop 10 Recommendations for Hypothetical User\n")
```

```
##
## Top 10 Recommendations for Hypothetical User
```

```
rec_newuser <- IBCF_recommend(
  newuser      = newuser,
  sim_matrix   = cosine_top30,
  movie_summary = movie_summary,
  top_n        = 10
)

print(rec_newuser)
```

```
##      Title PredictedRating
## 1      1                5
## 150    150               5
## 527    527               5
## 588    588               5
## 594    594               5
## 608    608               5
## 720    720               5
## 745    745               5
## 919    919               5
## 1097   1097              5
```